Control Flow

Naufal Rizqi Habibi
Muhammad Fikri Hidayat

Departemen Matematika
Institut Teknologi Sepuluh Nopember

Week 7

Masalah

Masalah

Dalam hidup, penting untuk memahami kapan waktunya **berhenti**, **melanjutkan**, atau **mengulang kembali** ke titik awal.

Hal tersebut perlu kita lakukan untuk mengontrol setiap langkah yang kita ambil. Jangan sampai kita terjebak dalam lingkaran (*loop*) yang sama selamanya, atau bahkan tidak tahu kapan harus melangkah ke jalan yang benar.

Solusi: Jump Statements

Di Java, kita mengontrol aliran tersebut menggunakan *Jump Statements*: `break`, `continue`, dan `return`.

Daftar isi

1. Break
2. Continue
3. Return
4. Ringkasan & Komparasi
5. Tugas Praktikum

Definisi

Break aadalah sebuah perintah yang digunakan untuk **menghentikan atau keluar secara paksa** dari sebuah perulangan (`for`, `while`, `do-while`) atau `switch-case` yang sedang berjalan. Biasanya `break` diletakkan di dalam blok `if` atau `else` untuk mengevaluasi kondisi berhentinya.

Listing 1: Contoh Penggunaan break dalam for

```
for (...) {
    ...
    if(...){
        break; // Keluar dari loop
    }
}
```

Break

Flowchart

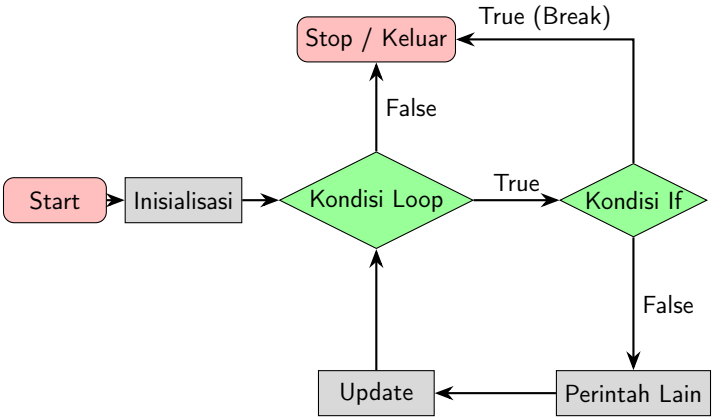


Figure: Alur Eksekusi break

Break

Contoh

Program untuk menentukan bilangan prima atau bilangan komposit.

```
int bilangan = input.nextInt();
for (int i = 2; i < bilangan; i++) {
    if (bilangan % i == 0) {
        System.out.println(bilangan + " bukan bilangan prima");
        break;
    }
    if (i == bilangan - 1) {
        System.out.println(bilangan + " adalah bilangan prima");
    }
}
```

```
Masukkan bilangan : 27
27 bukan bilangan prima
```

Label

Label adalah sebuah penanda yang diletakkan di suatu baris pada kode program yang berfungsi sebagai penanda untuk suatu blok kode tertentu. Label digunakan untuk mengidentifikasi blok kode yang akan diekseskusi oleh perintah break atau continue.

Listing 2: Contoh Label Break

```
outerloop:
for (...) {
    for (...) {
        if (...) {
            ...
            break outerloop; // Keluar dari semua loop di bawah label
        }
    }
}
```

Break

Perbedaan

`break` biasa hanya akan menghentikan satu perulangan saja, sedangkan `break` dengan label akan menghentikan perulangan yang diberi label (Bisa lebih dari satu perulangan, tergantung pada berapa perulangan yang berada dibawah label tersebut).

Listing 3: Contoh Non-Labeled Break

```
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (j == 1) {
            break; // Hanya menghentikan loop bagian dalam
        }
        System.out.println("i=" + i + ", j=" + j);
    }
}
```

```
i=0, j=0
i=1, j=0
i=2, j=0
```

Listing 4: Contoh Labeled Break

```
outerLoop: // Label untuk loop luar
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (j == 1) {
            break outerLoop; // Menghentikan loop luar
        }
        System.out.println("i=" + i + ", j=" + j);
    }
}
```

```
i = 0 , j = 0
```

Break

Live Code

Deskripsi Soal

Buatlah program untuk mencari bilangan pertama yang merupakan kelipatan dari angka A sekaligus kelipatan dari angka B, tetapi nilainya harus **lebih besar** dari suatu batas bawah X.

Hint

- Gunakan perulangan yang dimulai dari $X + 1$.
- Anda bisa menggunakan `while(true)` atau `for (int i = X + 1; ; i++)`.
- Gunakan `if` dengan kondisi `i % A == 0 && i % B == 0`, lalu cetak angka `i` dan panggil `break;`.

Contoh I/O

```
Masukkan A : 4
Masukkan B : 6
Batas X : 20

Output :
Bilangan ditemukan:  24
```

Daftar isi

1. Break
2. Continue
3. Return
4. Ringkasan & Komparasi
5. Tugas Praktikum

Continue

Definisi

Continue adalah sebuah perintah yang digunakan untuk melanjutkan perulangan berikutnya tanpa mengeksekusi kode di bawahnya. Berbeda dengan `break`, `continue` hanya memberhentikan atau melompati **1 iterasi saja**. Program akan tetap berjalan hingga salah satu kondisi untuk berhenti terpenuhi.

Listing 5: Contoh Penggunaan continue dalam for

```
for (int i = 0; i < batas; i++) {
    // ... kode awal ...
    if (kondisi_skip) {
        continue; // Lompati kode di bawah, lanjut ke iterasi berikutnya
    }
    // ... kode ini diabaikan HANYA untuk iterasi saat ini
}
```

Continue

Flowchart

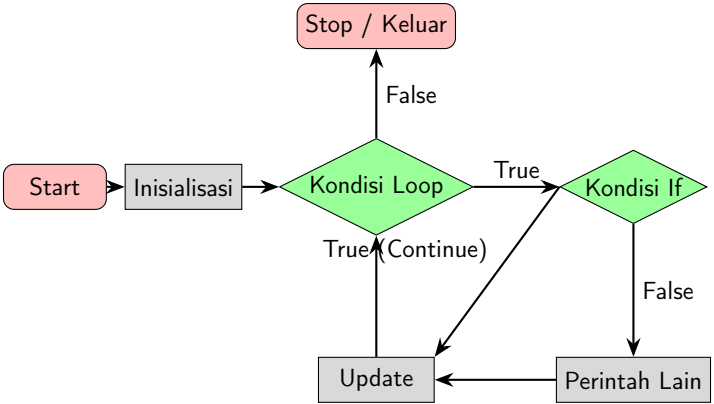


Figure: Alur Eksekusi Continue

Continue

Contoh

Program untuk menampilkan faktor dari sebuah bilangan.

```
for (int i = 1; i <= bilangan; i++) {
    // Jika sisa bagi bukan nol, lewati angka ini
    if (bilangan % i != 0) {
        continue;
    }
    // Cetak angka jika ia adalah faktor (sisa bagi 0)
    System.out.print(i + " ");
}
System.out.println("\nProgram selesai");
```

```
Masukkan bilangan: 12
1 2 3 4 6 12
Program selesai
```

Label

Sama seperti `break`, `continue` juga dapat diberi label. `continue` berlabel akan melompat langsung ke iterasi berikutnya dari *loop* bagian luar. *Namun, penggunaan continue dengan label tidak terlalu umum.*

Listing 6: Contoh Labeled Continue

```
outerLoop: // Label untuk loop luar
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (j == 1) {
            // Melompat ke iterasi berikutnya dari loop luar (i)
            continue outerLoop;
        }
        System.out.println("i=" + i + ", j=" + j);
    }
}
```

Deskripsi Soal

Buatlah program yang menerima input satu bilangan positif N. Program harus mencetak angka dari 1 sampai N, tetapi **lewati** angka yang merupakan kelipatan 3 **atau** kelipatan 5 menggunakan `continue`.

Hint

- Gunakan `for` loop dari 1 sampai N.
- Gunakan evaluasi logika `||` (OR) dengan operator modulus (%).

Contoh I/O

```
Masukkan N : 15
Output :  1 2 4 7 8 11 13 14
```

Daftar isi

1. Break
2. Continue
3. Return
4. Ringkasan & Komparasi
5. Tugas Praktikum

Definisi

Return adalah sebuah perintah yang digunakan untuk mengembalikan nilai atau **menghentikan proses dari suatu fungsi**. Karena kita masih belum membahas fungsi/method, intinya `return` digunakan untuk menghentikan semua proses yang ada di dalam method `main` (untuk sekarang).

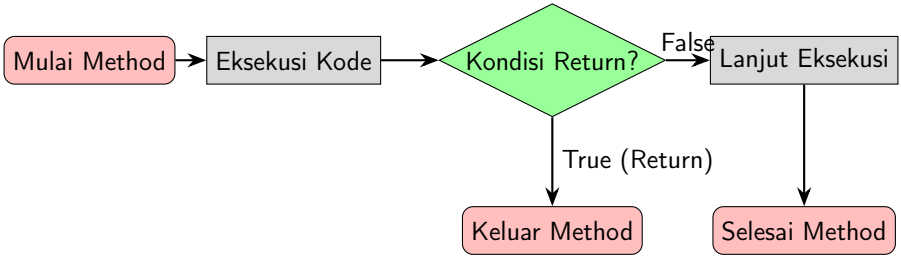


Figure: Alur Eksekusi Return

Return

Listing 7: Contoh penggunaan return

```
int x = input.nextInt();

    if (x % 2 == 0) {
        System.out.println("Bilangan genap");
        return; // Mengakhiri method main saat ini juga
    }

    System.out.println("Bilangan ganjil");
}
```

```
Masukkan bilangan: 12
Bilangan genap
```

```
Masukkan bilangan: 7
Bilangan ganjil
```

Daftar isi

1. Break
2. Continue
3. Return
4. Ringkasan & Komparasi
5. Tugas Praktikum

Ringkasan

Kapan Harus Memilih For, While, atau Do-While?

Kriteria	`break`	`continue`	`return`
Fungsi Utama	Menghentikan dan keluar dari blok eksekusi secara paksa.	Melewati sisa instruksi pada putaran (*iterasi*) saat ini.	Keluar dari *method* (fungsi) seutuhnya dan bisa mengembalikan nilai.
Ruang Lingkup	Berlaku di dalam *loop* (`for`, `while`, `do-while`) dan `switch`.	Hanya berlaku di dalam struktur *loop* (`for`, `while`, `do-while`).	Berlaku di dalam *method* (bisa dipanggil di dalam atau di luar *loop*).
Efek pada Loop	*Loop* langsung berhenti total. Program lanjut ke baris setelah *loop*.	*Loop* tetap berjalan, hanya melompat ke putaran berikutnya.	*Loop* berhenti total, dan seluruh fungsi ikut terhenti saat itu juga.
Tipe Berlabel (*Labeled*)	Bisa (*Labeled Break*) untuk keluar dari *nested loop* sekaligus.	Bisa (*Labeled Continue*) untuk lanjut ke iterasi *loop* terluar.	Tidak memiliki tipe berlabel.

Daftar isi

1. Break
2. Continue
3. Return
4. Ringkasan & Komparasi
5. Tugas Praktikum

Penugasan

Tugas praktikum minggu ini dapat diakses melalui tautan berikut:

Klik di sini untuk membuka lembar soal tugas